

As a Senior API Architect, I have designed the following RESTful API specification for your Banking Management System.

1. API Overview

- **Base URL:** `https://api.banking-system.com/v1`
- **Format:** JSON (UTF-8)
- **Versioning:** URI-based (`/v1/`)
- **Security:** OAuth 2.0 + OpenID Connect, with mandatory MFA header for sensitive operations.

2. Authentication

- **Protocol:** JWT (JSON Web Token) issued via OAuth 2.0 flow.
- **MFA Header:** Sensitive endpoints (transfers, loan applications) require an `X-MFA-Token` header containing a time-based one-time password (TOTP) or dynamic code.
- **Scopes:** `read:accounts`, `write:transfers`, `admin:audit`, `loan:apply`.

3. Endpoints

Method	Endpoint	Description	Access Level
POST	/auth/login	Authenticate user and retrieve JWT	Public
GET	/accounts/{id}	Get account balance and details	Customer
POST	/transactions	Initiate a bank transfer	Customer
GET	/loans/pending	List applications for approval	Staff
PATCH	/loans/{id}/status	Approve or Deny a loan application	Staff
GET	/analytics/spend	Retrieve AI-driven budget insights	Customer

| **GET** | `/admin/audit` | Retrieve system logs | Admin |

4. Sample Request/Response

POST /transactions (Initiating a Transfer)

Request Body:

```
{
  "src_account_id": "550e8400-e29b-41d4-a716-446655440000",
  "dest_account_id": "771e8400-e29b-41d4-a716-446655449999",
  "amount": 1500.00,
  "currency": "USD"
}
```

Response Body (202 Accepted):

```
{
  "txn_id": "a1b2c3d4-e5f6-7890",
  "status": "PENDING",
  "fraud_score": 12,
  "message": "Transaction initiated and under review."
}
```

5. Status Codes

- **200 OK:** Successful GET or PUT.
- **201 Created:** Successful resource creation (e.g., new account).
- **202 Accepted:** Request received, processing (e.g., pending fraud check).
- **400 Bad Request:** Schema validation error.
- **401 Unauthorized:** Missing/Invalid token.
- **403 Forbidden:** MFA required or insufficient role permissions.
- **429 Too Many Requests:** Rate limit exceeded.

6. Validation & Error Handling

We will use a standard Problem Details for HTTP APIs (RFC 7807) format.

Error Response Structure:

```
{
  "type": "https://api.banking.com/errors/insufficient-funds",
  "title": "Insufficient Balance",
  "status": 400,
  "detail": "Account balance is less than the requested transfer amount.",
  "instance": "/v1/transactions/txn_123"
}
```

Validation Requirements:

1. **Request Body:** All `amount` fields must be `decimal` or `long` (representing minor units) to avoid floating-point errors.
2. **MFA Check:** Middleware will intercept requests to sensitive endpoints. If `X-MFA-Token` is missing or invalid, a `403 Forbidden` with a `MFA\_REQUIRED` error code is returned.
3. **Audit Logging:** Every mutating request (POST/PUT/PATCH) triggers an asynchronous event that writes to the `Audit\_Logs` table.

7. Senior Architect Recommendations

1. **Idempotency:** For all transaction endpoints, enforce an `Idempotency-Key` header to prevent double-charging in case of network retries.
2. **Circuit Breakers:** Implement circuit breakers (e.g., Hystrix/Resilience4j) for the external credit bureau API integration. If the bureau is down, the system should gracefully transition to "Manual Review" rather than returning a 500 error.
3. **Rate Limiting:** Implement token-bucket rate limiting based on `user\_id` to prevent brute-force attacks on the transaction endpoints.
4. **Immutability:** Once a `Transaction` status reaches `COMPLETED`, ensure the record cannot be updated, only reversed by a new compensating transaction.